

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

NODE.JS MODULES: os, path AND fs (CO-2)

1. TITLE

What Are Node.js Modules? Explain the Purpose of the os, path, and fs Modules with Suitable Examples.

2. AIM

To study the concept of modularity in Node.js and to demonstrate the use of the built-in os, path, and fs modules for obtaining operating-system information, handling file paths, and performing file operations.

3. OBJECTIVE

- To understand what a module is and the types of modules in Node.js (core, local, and third-party).
- To load modules using require() and to understand the CommonJS module system.
- To use the os module for reading operating-system and hardware information.
- To use the path module for building, resolving, and analysing file paths in a platform-independent way.
- To use the fs module to create, read, append, rename, and delete files and folders.
- To differentiate between synchronous, callback-based, and promise-based file operations and to handle file errors.
- To combine the os, path, and fs modules in a single application.

4. THEORY

1. What is a Module in Node.js?

A module is a self-contained unit of code (usually a single JavaScript file) that has its own scope and exposes only the functionality that it wants to share. In Node.js every file is treated as a separate module, so the variables and functions declared in one file do not interfere with those of another file. Modules make a program **modular**: the code is divided into small, reusable, and easily maintainable parts instead of one large file.

Node.js uses the CommonJS module system. A module makes something available to others using `module.exports`, and another file loads it using `require()`. (Node.js also supports ES modules with `import` and `export`.)

2. Types of Modules

Type	Description	Example
Core (built-in)	Installed along with Node.js. Loaded by name; no installation is needed.	<code>os</code> , <code>path</code> , <code>fs</code> , <code>http</code> , <code>events</code>
Local (user-defined)	Created by the programmer as a separate <code>.js</code> file and loaded using its relative path.	<code>require('./math.js')</code>
Third-party	Written by others and installed using NPM into the <code>node_modules</code> folder.	<code>express</code> , <code>lodash</code> , <code>mongoose</code>

3. Creating and Loading a Module

```
// math.js (local module)
function add(a, b) { return a + b; }
module.exports = { add };

// app.js
const os = require('os');           // core module
const { add } = require('./math.js'); // local module
console.log(add(2, 3), os.platform());
```

4. The `os` Module

The **`os`** module provides information about the operating system and the hardware on which the program is running. It is used to write programs that adapt themselves to the platform, to monitor system resources such as memory and CPU, and to create diagnostic or system-report tools.

Method / Property	Purpose
<code>os.platform()</code>	Returns the platform, for example <code>'win32'</code> , <code>'linux'</code> , or <code>'darwin'</code> .
<code>os.type()</code> / <code>os.release()</code>	Name and release version of the operating system.
<code>os.arch()</code>	CPU architecture, for example <code>'x64'</code> or <code>'arm64'</code> .
<code>os.hostname()</code>	Name of the computer.
<code>os.homedir()</code> / <code>os.tmpdir()</code>	Home directory and temporary directory of the current user.
<code>os.cpus()</code>	Array describing every CPU core; its length is the number of cores.
<code>os.totalmem()</code> / <code>os.freemem()</code>	Total and free memory in bytes.
<code>os.uptime()</code>	Time since the system started, in seconds.
<code>os.userInfo()</code>	Details of the current user such as the username.
<code>os.EOL</code>	End-of-line marker: <code>'\n'</code> on Linux/macOS and <code>'\r\n'</code> on Windows.

5. The path Module

The **path** module provides utilities for working with file and directory paths. Paths are written differently on different systems (/ on Linux/macOS and \ on Windows), so building paths by joining strings is error-prone. The path module builds and analyses paths correctly for the operating system on which the program runs.

Method / Property	Purpose
path.basename(p [, ext])	Last part of the path (file name); the optional second argument removes the extension.
path.dirname(p)	The folder part of the path.
path.extname(p)	File extension including the dot, for example '.txt'.
path.parse(p) / path.format(obj)	Breaks a path into an object (root, dir, base, ext, name) / builds a path from such an object.
path.join(a, b, ...)	Joins the segments using the correct separator and resolves '.' and '..'.
path.resolve(a, b, ...)	Builds an absolute path, starting from the current working directory.
path.normalize(p)	Cleans a path by removing duplicate separators, '.', and '..'.
path.isAbsolute(p)	Returns true if the path is absolute.
path.sep	Path separator of the platform: '/' or '\\'.

6. The fs Module

The **fs** (file system) module is used to work with files and folders: to create, read, write, append, rename, and delete them. It offers three styles of operation:

- **Synchronous** (for example `readFileSync`): blocks the program until the operation is complete. It is simple and suitable for small scripts.
- **Asynchronous with callbacks** (for example `readFile(path, callback)`): does not block the program; the callback receives (err, data) when the operation finishes. This non-blocking style is preferred in servers.
- **Promise-based** (`require('fs/promises')`): used with `async/await` for cleaner code.

Method	Purpose
fs.existsSync(p)	Checks whether a file or folder exists.
fs.mkdirSync(p) / fs.rmdirSync(p)	Creates / removes a folder.
fs.writeFileSync(p, data)	Creates a file (or overwrites it) with the given data.
fs.appendFileSync(p, data)	Adds data at the end of a file.
fs.readFileSync(p, 'utf8')	Reads the complete file as text.
fs.statSync(p)	Returns file information such as size and <code>isFile()</code> .
fs.renameSync(old, new)	Renames or moves a file.
fs.readdirSync(p)	Lists the names of files and folders inside a folder.
fs.unlinkSync(p)	Deletes a file.

Method	Purpose
<code>fs.readFile(p, enc, cb)</code>	Asynchronous read; the callback receives (err, data).

7. Error Handling in fs

File operations can fail, for example when a file does not exist or permission is denied. Synchronous methods throw an exception (handled with try/catch), while callback methods pass an error object as the first argument. The error object contains a code such as ENOENT (no such file or directory) or EACCES (permission denied).

5. ALGORITHM / PROCEDURE

1. Start.
2. Open the terminal and verify that Node.js is installed by running `node -v`.
3. Create a project folder and the file `os_demo.js`. Load the `os` module using `require('os')` and print the platform, type, release, architecture, host name, directories, CPU cores, memory, uptime, user name, and EOL.
4. Create `path_demo.js`. Load the `path` module and apply `basename()`, `dirname()`, `extname()`, `parse()`, `join()`, `resolve()`, `normalize()`, `isAbsolute()`, `format()`, and `sep` on sample paths.
5. Create `fs_sync_demo.js`. Load the `fs` and `path` modules; create a folder, write a file, append a line, read the file, get its information, rename it, list the folder, and delete the file and the folder using synchronous methods.
6. Create `fs_async_demo.js`. Read a file using a callback, handle the error for a missing file, and read and append using `fs/promises` with `async/await`. Observe that the program continues without waiting for the file operation.
7. Create `report.js` that uses `os`, `path`, and `fs` together: collect the system information, save it in `reports/system-report.txt`, and read it back.
8. Run each file using `node` followed by the file name.
9. Observe the output and verify the file operations.
10. Stop.

6. PROGRAM

os_demo.js

```
// os_demo.js - operating system information using the built-in os module

// Step 1: Load the built-in os module
const os = require('os');

// Step 2: Operating system details
console.log('Platform      :', os.platform());
console.log('OS Type       :', os.type());
console.log('OS Release    :', os.release());
console.log('Architecture  :', os.arch());
console.log('Host Name     :', os.hostname());

// Step 3: Important directories
console.log('Home Directory  :', os.homedir());
console.log('Temp Directory  :', os.tmpdir());

// Step 4: Hardware details (CPU, memory, uptime)
console.log('CPU Cores       :', os.cpus().length);
console.log('Total Memory    :',
  (os.totalmem() / 1024 ** 3).toFixed(2), 'GB');
console.log('Free Memory     :',
  (os.freemem() / 1024 ** 3).toFixed(2), 'GB');
console.log('System Uptime   :',
  (os.uptime() / 3600).toFixed(2), 'hours');

// Step 5: User and line-ending details
console.log('User Name       :', os.userInfo().username);
console.log('End-of-Line (EOL) :', JSON.stringify(os.EOL));
```

path_demo.js

```
// path_demo.js - working with file paths using the built-in path module

// Step 1: Load the built-in path module
const path = require('path');

// Step 2: Take a sample path
const filePath = '/home/student/projects/node/notes.txt';

// Step 3: Split the path into its parts
console.log('Given path      :', filePath);
console.log('basename        :', path.basename(filePath));
console.log('basename (no ext) :', path.basename(filePath, '.txt'));
console.log('dirname         :', path.dirname(filePath));
console.log('extname         :', path.extname(filePath));
console.log('parse           :', path.parse(filePath));

// Step 4: Build and clean paths
console.log('join            :',
  path.join('folder', 'sub', '..', 'data', 'file.json'));
console.log('resolve         :', path.resolve('data', 'file.json'));
const messy = '/home//student/./projects/./notes.txt';
console.log('normalize       :', path.normalize(messy));

// Step 5: Check a path and create a path from parts
```

```

console.log('isAbsolute      :',
  path.isAbsolute(filePath), '/', path.isAbsolute('a.json'));
const parts = { dir: '/home/student', name: 'report', ext: '.pdf' };
console.log('format          :', path.format(parts));

// Step 6: Platform separator and current file
console.log('separator (sep)  :', path.sep);
console.log('current file     :', path.basename(__filename));

```

fs_sync_demo.js

// fs_sync_demo.js - synchronous file operations using the built-in fs module

```

// Step 1: Load the built-in fs and path modules
const fs = require('fs');
const path = require('path');

// Step 2: Define the folder and file paths
const dir = path.join(__dirname, 'data');
const file = path.join(dir, 'notes.txt');

// Step 3: Create the folder if it does not exist
if (!fs.existsSync(dir)) {
  fs.mkdirSync(dir);
  console.log('mkdirSync      : folder "data" created');
}

// Step 4: Create the file and write text into it
fs.writeFileSync(file, 'Node.js file system module.\n');
console.log('writeFileSync   : notes.txt created');

// Step 5: Add one more line at the end of the file
fs.appendFileSync(file, 'Line 2 added using appendFileSync.\n');
console.log('appendFileSync  : one more line added');

// Step 6: Read the file and display its content
console.log('readFileSync    : content of notes.txt ->');
console.log(fs.readFileSync(file, 'utf8'));

// Step 7: Get information about the file
const info = fs.statSync(file);
console.log('statSync       : size =', info.size, 'bytes | isFile =',
  info.isFile());

// Step 8: Rename the file and list the folder
fs.renameSync(file, path.join(dir, 'renamed.txt'));
console.log('renameSync     : notes.txt -> renamed.txt');
console.log('readdirSync    :', fs.readdirSync(dir));

// Step 9: Delete the file and then the folder
fs.unlinkSync(path.join(dir, 'renamed.txt'));
console.log('unlinkSync     : renamed.txt deleted');
fs.rmdirSync(dir);
console.log('rmdirSync      : folder "data" deleted');

```

fs_async_demo.js

```
// fs_async_demo.js - asynchronous file operations (callback and promise style)

// Step 1: Load the built-in modules
const fs = require('fs');
const fsp = require('fs/promises');
const path = require('path');

// Step 2: Create a file to read
const file = path.join(__dirname, 'async_notes.txt');
fs.writeFileSync(file, 'Content read asynchronously.');
```

console.log('1. Program started');

```
// Step 3: Read the file using a callback
fs.readFile(file, 'utf8', (err, data) => {
  if (err) return console.log('Callback error:', err.code);
  console.log('3. readFile (callback) ->', data);

  // Step 4: Handle the error when the file does not exist
  fs.readFile(path.join(__dirname, 'missing.txt'), 'utf8', (err2) => {
    console.log('4. Missing file handled ->', err2.code);
    readWithPromises();
  });
});

// Step 5: Append, read, and delete using promises with async/await
async function readWithPromises() {
  await fsp.appendFile(file, ' Appended using fs/promises.');
```

const data = await fsp.readFile(file, 'utf8');

console.log('5. fs/promises ->', data);

await fsp.unlink(file);

console.log('6. async_notes.txt deleted');

}

```
// Step 6: This line runs before the file content is printed (non-blocking)
console.log('2. Program continues without waiting for the file operation');
```

report.js

```
// report.js - uses os, path and fs together to create a system report file

// Step 1: Load the built-in modules
const os = require('os');
const path = require('path');
const fs = require('fs');
```

```
// Step 2: Define the report folder and file paths
const reportDir = path.join(__dirname, 'reports');
const reportFile = path.join(reportDir, 'system-report.txt');
```

```
// Step 3: Create the folder if it does not exist
if (!fs.existsSync(reportDir)) fs.mkdirSync(reportDir);

// Step 4: Collect the report lines (os and path information)
const lines = [
  'SYSTEM REPORT',
  '-----',
```

```
`Generated On : ${new Date().toISOString}`,  
`Platform      : ${os.platform()} (${os.arch()})`,  
`Host Name     : ${os.hostname()}`,  
`CPU Cores     : ${os.cpus().length}`,  
`Total Memory  : ${(os.totalmem() / 1024 ** 3).toFixed(2)} GB`,  
`Report File   : ${path.basename(reportFile)}`,  
`Saved In      : ${path.dirname(reportFile)}`,  
];
```

```
// Step 5: Write the report to the file  
fs.writeFileSync(reportFile, lines.join(os.EOL));  
console.log('Report written to :', reportFile);  
  
// Step 6: Read the report back and display it  
console.log(`\n----- Reading the report back -----`);  
console.log(fs.readFileSync(reportFile, 'utf8'));
```

7. CODE EXPLANATION

Code	Explanation
<code>const os = require('os');</code>	Loads the built-in os module. Core modules are loaded by name without a path.
<code>os.platform(), os.type(), os.arch()</code>	Return the platform, the name of the operating system, and the CPU architecture.
<code>os.totalmem() / 1024 ** 3</code>	Converts the total memory from bytes to gigabytes.
<code>os.cpus().length</code>	Number of CPU cores available.
<code>os.EOL</code>	Line-ending character(s) of the platform.
<code>const path = require('path');</code>	Loads the built-in path module.
<code>path.basename(filePath, '.txt')</code>	Returns the file name without the extension.
<code>path.parse(filePath)</code>	Splits the path into an object { root, dir, base, ext, name }.
<code>path.join('folder', 'sub', '..', 'data', 'file.json')</code>	Joins the segments with the correct separator and resolves '..', giving folder/data/file.json.
<code>path.resolve('data', 'file.json')</code>	Builds an absolute path starting from the current working directory.
<code>path.normalize(messy)</code>	Removes duplicate separators and resolves '.' and '..'.
<code>__dirname / __filename</code>	Absolute path of the folder / file of the current module.
<code>fs.existsSync(dir) / fs.mkdirSync(dir)</code>	Checks whether the folder exists and creates it if it does not.
<code>fs.writeFileSync(file, text)</code>	Creates the file and writes the text (overwrites the file if it exists).
<code>fs.appendFileSync(file, text)</code>	Adds text at the end of the file without removing the existing content.
<code>fs.readFileSync(file, 'utf8')</code>	Reads the whole file and returns its content as text.
<code>fs.statSync(file)</code>	Returns an information object; size gives bytes and isFile() gives true for files.
<code>fs.renameSync(old, new)</code>	Renames the file.
<code>fs.readdirSync(dir)</code>	Returns an array with the names of the items inside the folder.
<code>fs.unlinkSync(file) / fs.rmdirSync(dir)</code>	Deletes the file / the empty folder.
<code>fs.readFile(file, 'utf8', (err, data) => {...})</code>	Asynchronous read. The callback runs after the read is finished; the program continues in the meantime.
<code>err.code</code>	Holds the error code, for example 'ENOENT' when the file does not exist.
<code>require('fs/promises'); await fsp.readFile(...)</code>	Promise-based API used with async/await.
<code>lines.join(os.EOL)</code>	Joins the report lines using the line ending of the platform.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the programs in Node.js.

Test Case TC-01

Test / Input	Run node os_demo.js.
Expected Result	Information about the operating system and hardware (platform, type, release, architecture, host name, directories, CPU cores, memory, uptime, user name, and EOL) is displayed.
Actual Result	All the details were displayed correctly for the system on which the program was executed. The memory values were converted to GB and the EOL character was shown as "\n".
Status	Pass

TC-01 Output:

```
> C:\Windows\System32\cmd.exe
C:\FSWD\Exp6>node os_demo.js
Platform      : linux
OS Type       : Linux
OS Release    : 6.18.44-fc-v37
Architecture  : x64
Host Name     : vm
Home Directory : /root
Temp Directory : /tmp
CPU Cores     : 1
Total Memory  : 3.91 GB
Free Memory   : 3.62 GB
System Uptime : 0.15 hours
User Name     : root
End-of-Line (EOL) : "\n"
C:\FSWD\Exp6>_
```

Test Case TC-02

Test / Input	Run node path_demo.js.
Expected Result	The file name, folder, and extension are extracted; the path is parsed into an object; join, resolve, and normalize return clean paths; isAbsolute and format work correctly.
Actual Result	basename returned notes.txt (notes without extension), dirname returned /home/student/projects/node, extname returned .txt, join returned folder/data/file.json, normalize returned /home/student/notes.txt, isAbsolute returned true / false, and format returned /home/student/report.pdf.
Status	Pass

TC-02 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp6>node path_demo.js
Given path      : /home/student/projects/node/notes.txt
basename       : notes.txt
basename (no ext) : notes
dirname        : /home/student/projects/node
extname        : .txt
parse         : {
  root: '/',
  dir: '/home/student/projects/node',
  base: 'notes.txt',
  ext: '.txt',
  name: 'notes'
}
join           : folder/data/file.json
resolve        : /home/claude/lab/exp6/data/file.json
normalize      : /home/student/notes.txt
isAbsolute     : true / false
format         : /home/student/report.pdf
separator (sep) : /
current file    : path_demo.js
C:\FSWD\Exp6>_
```

Test Case TC-03

Test / Input	Run node fs_sync_demo.js.
Expected Result	A folder and a file are created, a line is appended, the two-line content is read, the file size is reported, the file is renamed and listed, and finally the file and the folder are deleted.
Actual Result	All operations were completed in sequence. The content showed both lines, statSync reported 63 bytes, readdirSync returned ['renamed.txt'], and the file and the folder were removed.
Status	Pass

TC-03 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp6>node fs_sync_demo.js
mkdirSync      : folder "data" created
writeFileSync   : notes.txt created
appendFileSync  : one more line added
readFileSync     : content of notes.txt ->
Node.js file system module.
Line 2 added using appendFileSync.

statSync       : size = 63 bytes | isFile = true
renameSync     : notes.txt -> renamed.txt
readdirSync    : [ 'renamed.txt' ]
unlinkSync     : renamed.txt deleted
rmdirSync      : folder "data" deleted
C:\FSWD\Exp6>_
```

Test Case TC-04

Test / Input	Run node fs_async_demo.js.
Expected Result	Message 2 is printed before the file content because the read is non-blocking. A missing file gives the code ENOENT. The promise version appends, reads, and deletes the file.
Actual Result	The messages appeared in the order 1, 2, 3, 4, 5, 6. This confirmed that the program continued without waiting, the missing file was handled with ENOENT, and fs/promises worked with async/await.
Status	Pass

TC-04 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp6>node fs_async_demo.js
1. Program started
2. Program continues without waiting for the file operation
3. readFile (callback) -> Content read asynchronously.
4. Missing file handled -> ENOENT
5. fs/promises -> Content read asynchronously. Appended using fs/promises.
6. async_notes.txt deleted
C:\FSWD\Exp6>_
```

Test Case TC-05

Test / Input	Run node report.js.
Expected Result	The reports folder and the file system-report.txt are created using os, path, and fs together, and the content of the file is read back and displayed.
Actual Result	The report was written and its content was displayed: title, generation time, platform, host name, CPU cores, total memory, file name, and folder.
Status	Pass

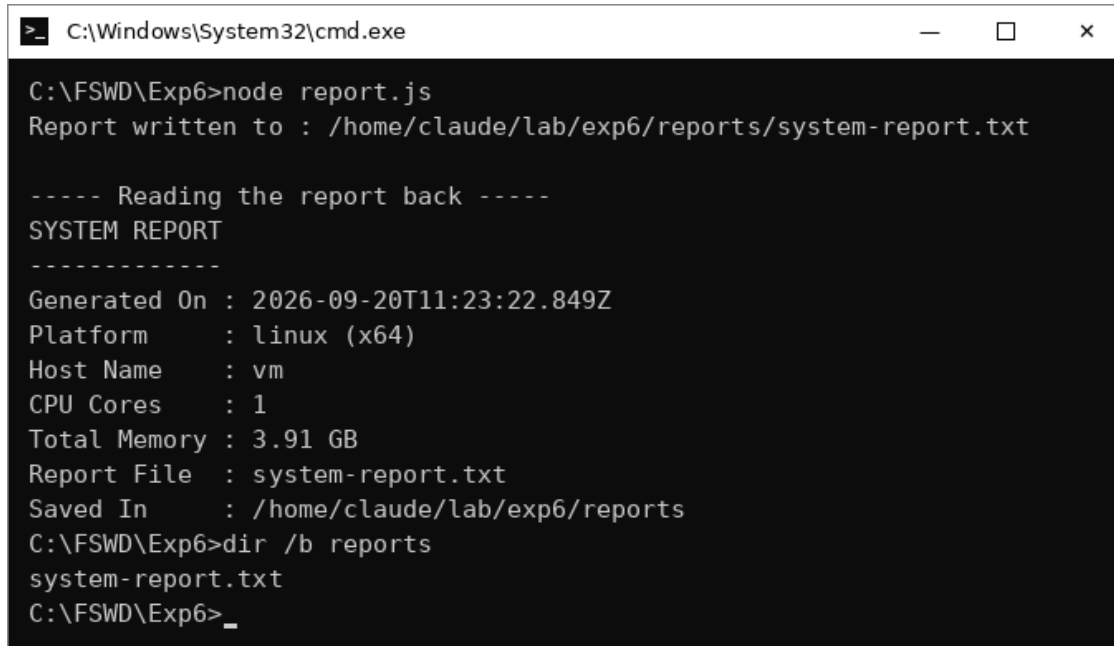
TC-05 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp6>node report.js
Report written to : /home/claude/lab/exp6/reports/system-report.txt

----- Reading the report back -----
SYSTEM REPORT
-----
Generated On : 2026-09-20T11:23:22.808Z
Platform      : linux (x64)
Host Name     : vm
CPU Cores     : 1
Total Memory  : 3.91 GB
Report File   : system-report.txt
Saved In      : /home/claude/lab/exp6/reports
C:\FSWD\Exp6>_
```

9. OUTPUT

Output 1: Execution of report.js (os, path, and fs used together) and the generated folder



```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp6>node report.js
Report written to : /home/claude/lab/exp6/reports/system-report.txt

----- Reading the report back -----
SYSTEM REPORT
-----
Generated On : 2026-09-20T11:23:22.849Z
Platform      : linux (x64)
Host Name     : vm
CPU Cores     : 1
Total Memory  : 3.91 GB
Report File   : system-report.txt
Saved In      : /home/claude/lab/exp6/reports
C:\FSWD\Exp6>dir /b reports
system-report.txt
C:\FSWD\Exp6>_
```

Figure 1: Output of node report.js and the contents of the reports folder

10. RESULT

Thus, the concept of modules in Node.js was studied and the built-in **os**, **path**, and **fs** modules were used successfully. The **os** module provided system information, the **path** module handled file paths independently of the platform, and the **fs** module performed file creation, reading, appending, renaming, and deletion in synchronous, callback, and promise styles. The programs executed without errors, and the missing-file error was handled using the error code ENOENT.